

Paralelização do Conjunto de Mandelbrot com MPI no Modelo Coordenador/Trabalhador

Bernardo Luiz Padilha Zamin · João Pedro Sostruznik Sotero da Cunha
Programação Paralela — Trabalho 2 (MPI) · PUCRS · 2026

1. Introdução

O problema escolhido é o cálculo do **conjunto de Mandelbrot**, reaproveitado do primeiro trabalho (OpenMP). Para cada pixel (x,y) de uma imagem de 800×600 , mapeia-se um ponto c do plano complexo e itera-se $z = z^2 + c$ até $|z| > 2$ ou atingir `MAX_ITER`; o número de iterações define a cor. O custo por pixel é **altamente irregular** — pontos dentro do conjunto executam todas as `MAX_ITER` iterações, enquanto pontos externos escapam cedo — o que torna o **balanceamento de carga** o ponto central do problema.

2. Modelo coordenador/trabalhador

A versão paralela foi implementada em C + MPI no padrão SPMD seguindo o modelo mestre/escravo. O **saco de trabalho** é a imagem, dividida em blocos de `rows_per_task = 10` linhas (unidades de trabalho). O processo de rank 0 é o **coordenador**; os ranks `1..n-1` são **trabalhadores**. A **iniciativa é dos trabalhadores**: cada um envia `TAG_REQUEST` pedindo trabalho; o coordenador responde com um bloco (`TAG_TASK`) enquanto houver linhas, ou com `TAG_STOP` quando o saco esvazia. O trabalhador calcula o bloco e devolve o resultado (`TAG_RESULT`: metadados + vetor de pixels). O coordenador grava cada bloco em `image[start_row*WIDTH]`, **preservando a ordem original** dos vetores independentemente da ordem de chegada (recepção por ordem de chegada via `MPI_Probe / MPI_ANY_SOURCE`). O programa encerra quando todos os trabalhadores recebem `STOP`. Verificamos que a imagem gerada é **idêntica** (mesmo checksum) para qualquer número de trabalhadores e igual à da versão sequencial, confirmando a corretude e a preservação da ordem.

3. Metodologia

A versão sequencial (referência) executa em um único processo. As medições paralelas foram feitas em **2 nós exclusivos da máquina atlântica** (LAD), variando de 1 a 15 trabalhadores (até 16 processos, 1 nó) e 31 trabalhadores (32 processos, 2 nós). Caso de teste: imagem 800×600 . Na **escala forte**, `MAX_ITER` = 10000

é fixo. Na **escala fraca**, a carga cresce proporcionalmente ao número de trabalhadores (`MAX_ITER` $\propto$ N), mantendo a carga por trabalhador constante. O tempo de referência é o de 1 trabalhador, no qual o coordenador não computa — equivalente à execução serial. $\text{Speed-up} = T_{\text{seq}} / T_p$ e $\text{Eficiência} = \text{Speed-up} / p$.

4. Resultados e análise

Escala forte. O speed-up cresce de forma quase linear até ~ 5 trabalhadores e segue subindo, atingindo $10,7\times$ com 15 trabalhadores e $20,2\times$ com 31. A eficiência cai de 0,97 (2 trab.) para 0,72 (15) e 0,65 (31). A perda decorre de: (i) o coordenador ser um processo serial que não computa e centraliza a recepção, tornando-se gargalo conforme mais trabalhadores disputam atendimento; (ii) crescimento do overhead de comunicação; (iii) efeito de cauda — com 60 blocos, ao aumentar os trabalhadores cresce a ociosidade no fim do processamento.

Balanceamento de carga. A distribuição é **dinâmica e orientada por demanda**: trabalhadores ociosos pedem o próximo bloco, o que absorve naturalmente a forte irregularidade de custo entre as linhas do Mandelbrot. Blocos de 10 linhas equilibram overhead de mensagens e granularidade. O resultado determinístico confirma que a ordem é mantida apesar do não-determinismo de chegada.

Escala fraca. Idealmente o tempo permaneceria constante ao escalar carga e trabalhadores juntos. O tempo medido sobe pouco ($10,8\text{ s} \rightarrow 16,2\text{ s}$ de 1 a 31 trab.) e a eficiência fraca cai de 1,0 para 0,66 — boa escalabilidade fraca, com a mesma degradação suave atribuída ao gargalo do coordenador e à comunicação.

5. Conclusão

O modelo coordenador/trabalhador com iniciativa dos trabalhadores forneceu bom speed-up e balanceamento de carga para um problema fortemente irregular, escalando até 32 processos em 2 nós. As limitações de eficiência vêm da serialização no coordenador e do overhead de comunicação, esperados nesse padrão; blocos adaptativos ou múltiplos coordenadores poderiam mitigá-las.

Anexo I — Tabelas e Gráficos

Tabela 1 — Escala forte (800×600, MAX_ITER=10000)

Trab.	Tempo (s)	Speed-up	Efic.
1	10,861261	1,00	1,000
2	5,596511	1,94	0,970
3	4,179426	2,60	0,866
4	3,255884	3,34	0,834
5	2,545801	4,27	0,853
6	2,072567	5,24	0,873
7	1,863472	5,83	0,833
8	1,657298	6,55	0,819
9	1,483067	7,32	0,814
10	1,384045	7,85	0,785
11	1,312784	8,27	0,752
12	1,194458	9,09	0,758
13	1,067153	10,18	0,783
14	1,033584	10,51	0,751
15	1,011426	10,74	0,716
31	0,537922	20,19	0,651

Tabela 2 — Escala fraca (carga ∝ N)

Trab.	Tempo (s)	Speed-up	Efic.
1	10,785180	1,00	1,000
2	11,220187	1,93	0,963
3	12,804295	2,51	0,837
4	13,448854	3,18	0,796
5	13,074383	4,09	0,818
6	12,586510	5,09	0,849
7	13,399998	5,59	0,798
8	13,823886	6,18	0,773
9	13,787825	6,97	0,775
10	14,119740	7,57	0,757
11	14,559463	8,08	0,734
12	14,021968	9,15	0,762
13	14,563222	9,53	0,733
14	14,665052	10,19	0,728
15	15,440807	10,37	0,691
31	16,232599	20,39	0,658

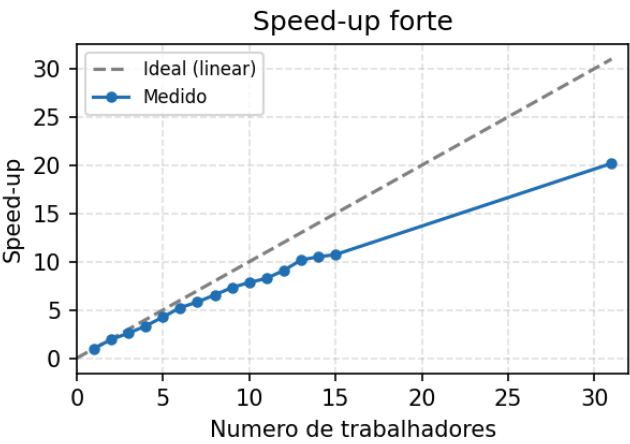


Fig. 1 — Speed-up (escala forte)

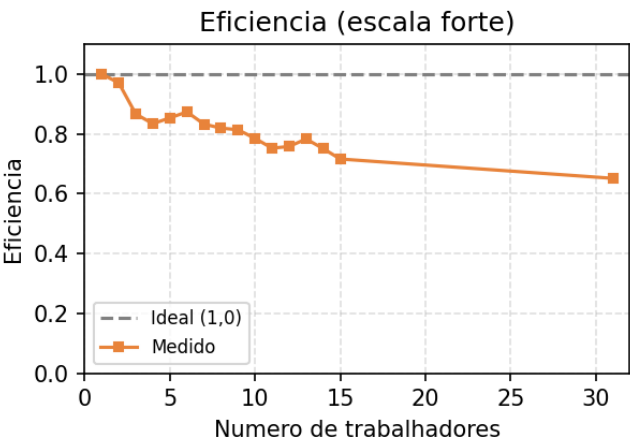


Fig. 2 — Eficiência (escala forte)

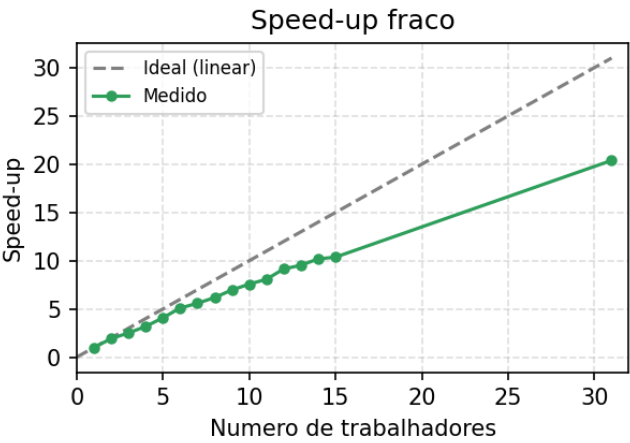


Fig. 3 — Speed-up (escala fraca)

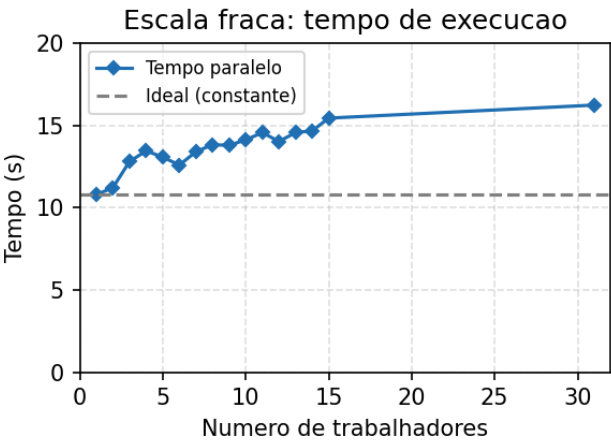


Fig. 4 — Tempo de execução (escala fraca)

Anexo II — Código-fonte

mandelbrot-seq.c — versão sequencial (referência)

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define WIDTH 800
#define HEIGHT 600

/*
 * Versao sequencial (sem MPI) usada como referencia (baseline) para o
 * calculo de speed-up e eficiencia. Faz exatamente o mesmo calculo da
 * versao paralela (mesmo mapeamento de coordenadas e mesmo algoritmo),
 * porem em um unico processo, percorrendo todas as linhas da imagem.
 */

int mandelbrot(double real, double imag, int max_iter)
{
    double z_real = 0.0;
    double z_imag = 0.0;

    int iter = 0;

    while ((z_real * z_real + z_imag * z_imag <= 4.0) &&
           iter < max_iter)
    {
        double temp = z_real * z_real - z_imag * z_imag + real;

        z_imag = 2.0 * z_real * z_imag + imag;
        z_real = temp;

        iter++;
    }

    return iter;
}

void save_ppm(const char *filename, int *image, int max_iter)
{
    FILE *fp = fopen(filename, "w");

    if (fp == NULL)
    {
        printf("Erro ao criar o arquivo %s\n", filename);
        return;
    }

    fprintf(fp, "P3\n");
    fprintf(fp, "%d %d\n", WIDTH, HEIGHT);
    fprintf(fp, "255\n");

    for (int i = 0; i < WIDTH * HEIGHT; i++)
    {
        int color = (image[i] * 255) / max_iter;

        fprintf(fp, "%d %d %d ",
                color,
                color,
                color);

        if ((i + 1) % WIDTH == 0)
            fprintf(fp, "\n");
    }

    fclose(fp);
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Uso: %s <max_iter>\n", argv[0]);
        return 1;
    }

    int max_iter = atoi(argv[1]);

    int *image = malloc(sizeof(int) * WIDTH * HEIGHT);

    if (image == NULL)
    {
        printf("Erro ao alocar memoria para a imagem.\n");
        return 1;
    }

    struct timespec t_start, t_end;
    clock_gettime(CLOCK_MONOTONIC, &t_start);
```

```

// Percorre todas as linhas da imagem em um unico processo.
for (int y = 0; y < HEIGHT; y++)
{
    for (int x = 0; x < WIDTH; x++)
    {
        double real = -2.5 + (3.5 * x / WIDTH);
        double imag = -1.0 + (2.0 * y / HEIGHT);

        image[y * WIDTH + x] = mandelbrot(real, imag, max_iter);
    }
}

clock_gettime(CLOCK_MONOTONIC, &t_end);

double elapsed = (t_end.tv_sec - t_start.tv_sec) +
    (t_end.tv_nsec - t_start.tv_nsec) / 1e9;

save_ppm("mandelbrot.ppm", image, max_iter);

free(image);

printf("Imagem salva em mandelbrot.ppm\n");
printf("Tempo total: %f segundos\n", elapsed);

return 0;
}

```

mandelbrot-v2.c — versão paralela MPI (iniciativa dos trabalhadores)

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define WIDTH 800
#define HEIGHT 600
#define MAX_ITER 10000

#define TAG_REQUEST 0
#define TAG_TASK 1
#define TAG_RESULT 2
#define TAG_STOP 3

typedef struct
{
    int start_row;
    int num_rows;
} Task;

int mandelbrot(double real, double imag, int max_iter)
{
    double z_real = 0.0;
    double z_imag = 0.0;

    int iter = 0;

    while ((z_real * z_real + z_imag * z_imag <= 4.0) &&
        iter < max_iter)
    {
        double temp = z_real * z_real - z_imag * z_imag + real;

        z_imag = 2.0 * z_real * z_imag + imag;
        z_real = temp;

        iter++;
    }

    return iter;
}

void save_ppm(const char *filename, int *image, int max_iter)
{
    FILE *fp = fopen(filename, "w");

    if (fp == NULL)
    {
        printf("Erro ao criar o arquivo %s\n", filename);
        return;
    }

    fprintf(fp, "P3\n");
    fprintf(fp, "%d %d\n", WIDTH, HEIGHT);
    fprintf(fp, "255\n");

    for (int i = 0; i < WIDTH * HEIGHT; i++)
    {
        int color = (image[i] * 255) / max_iter;

        fprintf(fp, "%d %d %d ",
            color,
            color,
            color,

```

```

        color);

    if ((i + 1) % WIDTH == 0)
        fprintf(fp, "\n");
}

fclose(fp);
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        return 1;
    }

    int max_iter = atoi(argv[1]);

    MPI_Init(&argc, &argv);

    int rank, size;
    double start, end;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2)
    {
        if (rank == 0)
            printf("Execute com pelo menos 2 processos: 1 coordenador e 1 trabalhador.\n");

        MPI_Finalize();
        return 0;
    }

    // =====
    // COORDENADOR
    // =====

    if (rank == 0)
    {
        start = MPI_Wtime();

        int *image = malloc(sizeof(int) * WIDTH * HEIGHT);

        if (image == NULL)
        {
            printf("Erro ao alocar memoria para a imagem.\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }

        int next_row = 0;
        int rows_completed = 0;
        int rows_per_task = 10;
        int workers_stopped = 0;

        MPI_Status status;

        /*
         * Agora a iniciativa e dos trabalhadores:
         * - o trabalhador envia TAG_REQUEST pedindo trabalho;
         * - o coordenador responde com TAG_TASK ou TAG_STOP;
         * - depois o trabalhador envia TAG_RESULT com o resultado.
         */
        while (workers_stopped < size - 1)
        {
            MPI_Probe(MPI_ANY_SOURCE,
                      MPI_ANY_TAG,
                      MPI_COMM_WORLD,
                      &status);

            int worker = status.MPI_SOURCE;

            if (status.MPI_TAG == TAG_REQUEST)
            {
                int request;

                MPI_Recv(&request,
                        1,
                        MPI_INT,
                        worker,
                        TAG_REQUEST,
                        MPI_COMM_WORLD,
                        MPI_STATUS_IGNORE);

                if (next_row < HEIGHT)
                {
                    Task task;

                    task.start_row = next_row;
                    task.num_rows = rows_per_task;

```

```

        if (next_row + rows_per_task > HEIGHT)
            task.num_rows = HEIGHT - next_row;

        MPI_Send(&task,
                 sizeof(Task),
                 MPI_BYTE,
                 worker,
                 TAG_TASK,
                 MPI_COMM_WORLD);

        next_row += task.num_rows;
    }
    else
    {
        Task stop_task;
        stop_task.start_row = -1;
        stop_task.num_rows = 0;

        MPI_Send(&stop_task,
                 sizeof(Task),
                 MPI_BYTE,
                 worker,
                 TAG_STOP,
                 MPI_COMM_WORLD);

        workers_stopped++;
    }
}
else if (status.MPI_TAG == TAG_RESULT)
{
    Task result_task;

    MPI_Recv(&result_task,
             sizeof(Task),
             MPI_BYTE,
             worker,
             TAG_RESULT,
             MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);

    int pixels = result_task.num_rows * WIDTH;

    MPI_Recv(&image[result_task.start_row * WIDTH],
             pixels,
             MPI_INT,
             worker,
             TAG_RESULT,
             MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);

    rows_completed += result_task.num_rows;
}
}

end = MPI_Wtime();

save_ppm("mandelbrot.ppm", image, max_iter);

free(image);

printf("Imagem salva em mandelbrot.ppm\n");
printf("Linhas calculadas: %d\n", rows_completed);
printf("Tempo total: %f segundos\n", end - start);
}

// =====
// WORKERS
// =====

else
{
    MPI_Status status;
    int request = 1;

    while (1)
    {
        // Trabalhador toma a iniciativa e pede trabalho ao coordenador.
        MPI_Send(&request,
                 1,
                 MPI_INT,
                 0,
                 TAG_REQUEST,
                 MPI_COMM_WORLD);

        Task task;

        MPI_Recv(&task,
                 sizeof(Task),
                 MPI_BYTE,
                 0,
                 MPI_ANY_TAG,
                 MPI_COMM_WORLD,

```

```

        &status);

    if (status.MPI_TAG == TAG_STOP)
        break;

    int *buffer = malloc(sizeof(int) * task.num_rows * WIDTH);

    if (buffer == NULL)
    {
        printf("Processo %d: erro ao alocar memoria para o buffer.\n", rank);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    for (int row = 0; row < task.num_rows; row++)
    {
        int y = task.start_row + row;

        for (int x = 0; x < WIDTH; x++)
        {
            double real = -2.5 + (3.5 * x / WIDTH);
            double imag = -1.0 + (2.0 * y / HEIGHT);

            int iter = mandelbrot(real, imag, max_iter);

            buffer[row * WIDTH + x] = iter;
        }
    }

    MPI_Send(&task,
            sizeof(Task),
            MPI_BYTE,
            0,
            TAG_RESULT,
            MPI_COMM_WORLD);

    MPI_Send(buffer,
            task.num_rows * WIDTH,
            MPI_INT,
            0,
            TAG_RESULT,
            MPI_COMM_WORLD);

    free(buffer);
}

MPI_Finalize();

return 0;
}

```